CSV 493

LINUX SYSTEM ADMINSTRATION

Unit – II

Controlling Access to Files with Linux File System Permissions : Linux File System Permissions, Managing File System Permissions from the Command Line, Managing Default Permissions and File Access

Monitoring and Managing Linux Processes : Processes, Controlling Jobs, Killing Processes, Monitoring Process Activity

Controlling Services and Daemons : Identifying Automatically Started System Processes, Controlling System Services

Configuring and Securing OpenSSH Service : Accessing the Remote Command Line with SSH, Configuring SSH Key-based Authentication, Customizing SSH Service Configuration

Analyzing and Storing Logs : System Log Architecture, Reviewing Sys log Files, Reviewing systemd Journal Entries, Preserving the systemd Journal, Maintaining Accurate Time

CONTROLLING ACCESS TO FILES

File permissions control access to files. The Linux file permissions system is simple but flexible, which makes it easy to understand and apply, yet still able to handle most normal permission cases easily.

Files have three categories of user to which permissions apply. The file is owned by a user, normally the one who created the file. The file is also owned by a single group, usually the primary group of the user who created the file, but this can be changed.

The most specific permissions take precedence. User permissions override group permissions, which override other permissions

CONTROLLING ACCESS TO FILES

Effects of Permissions on Files and Directories

Permission	Effect on Files	Effect on Directories
r (Read)	Contents of the file can be read.	Contents of the directory (the file names) can be listed.
w (Write)	Contents of the file can be changed.	Files in the directory can be created, deleted, or renamed (provided the user also has execute (x) permission on the directory).
x (Execute)	File can be executed as a command or program.	Directory can be accessed. You can cd into the directory, access its files, and retrieve information about them (subject to the files' own permissions).

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

```
[user@host~]$ ls -l test
-rw-rw-r--. 1 student student 0 Feb  8 17:36 test
```

The **-l** option of the **ls** command shows more detailed information about file permissions and ownership

```
[user@host ~]$ ls -ld /home
drwxr-xr-x. 5 root root 4096 Jan 31 22:00 /home
```

The first character of the long listing is the file type. You interpret it like this:

- **-** is a regular file.
- **d** is a directory.
- **l** is a soft link.

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

```
[user@host~]$ ls -l test
-rw-rw-r--. 1 student student 0 Feb  8 17:36 test
```

The **-l** option of the **ls** command shows more detailed information about file permissions and ownership

```
[user@host ~]$ ls -ld /home
drwxr-xr-x. 5 root root 4096 Jan 31 22:00 /home
```

The first character of the long listing is the file type. You interpret it like this:

- **-** is a regular file.
- **d** is a directory.
- **l** is a soft link.

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

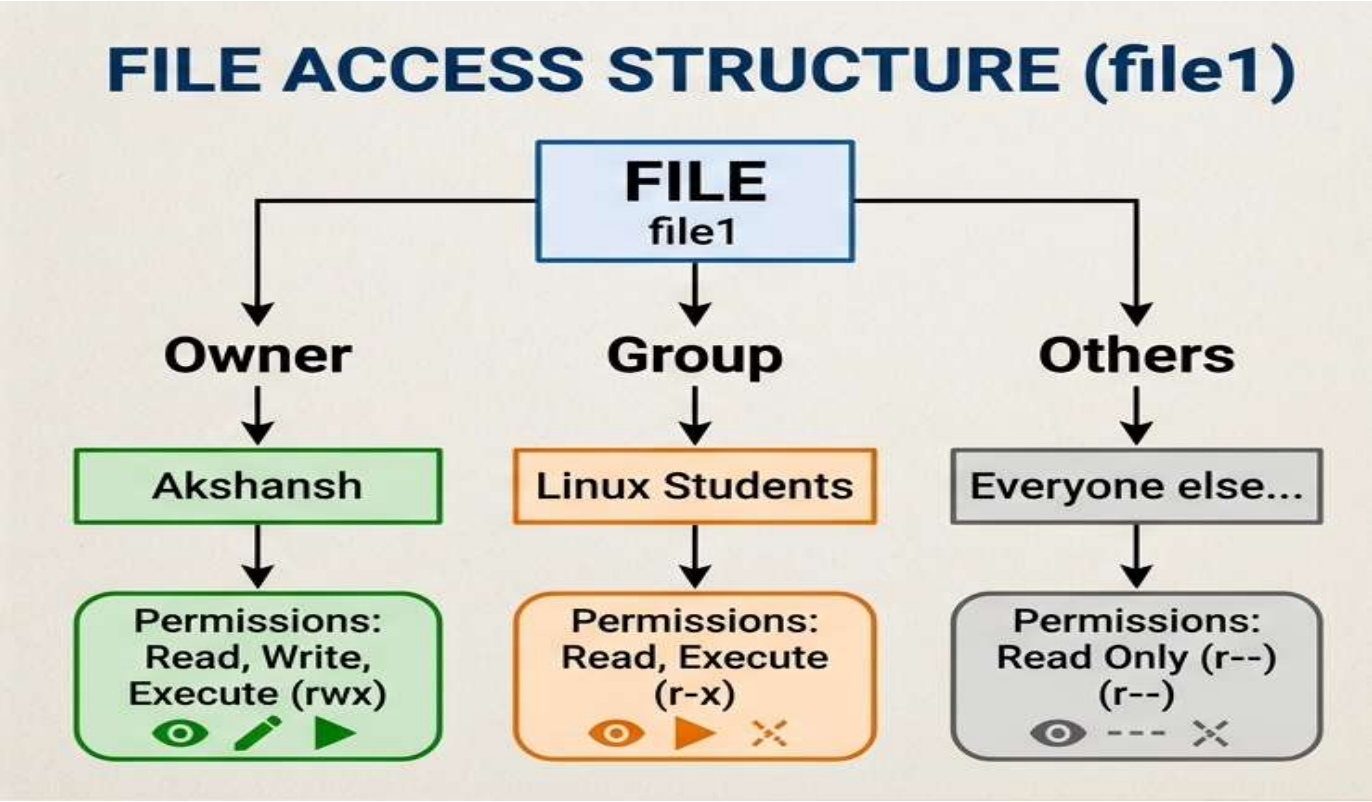
Changing Permissions with the Symbolic Method

```
chmod WhoWhatWhich file|directory
```

- Who is u, g, o, a (for user, group, other, all)
- What is +, -, = (for add, remove, set exactly)
- Which is r, w, x (for read, write, execute)

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

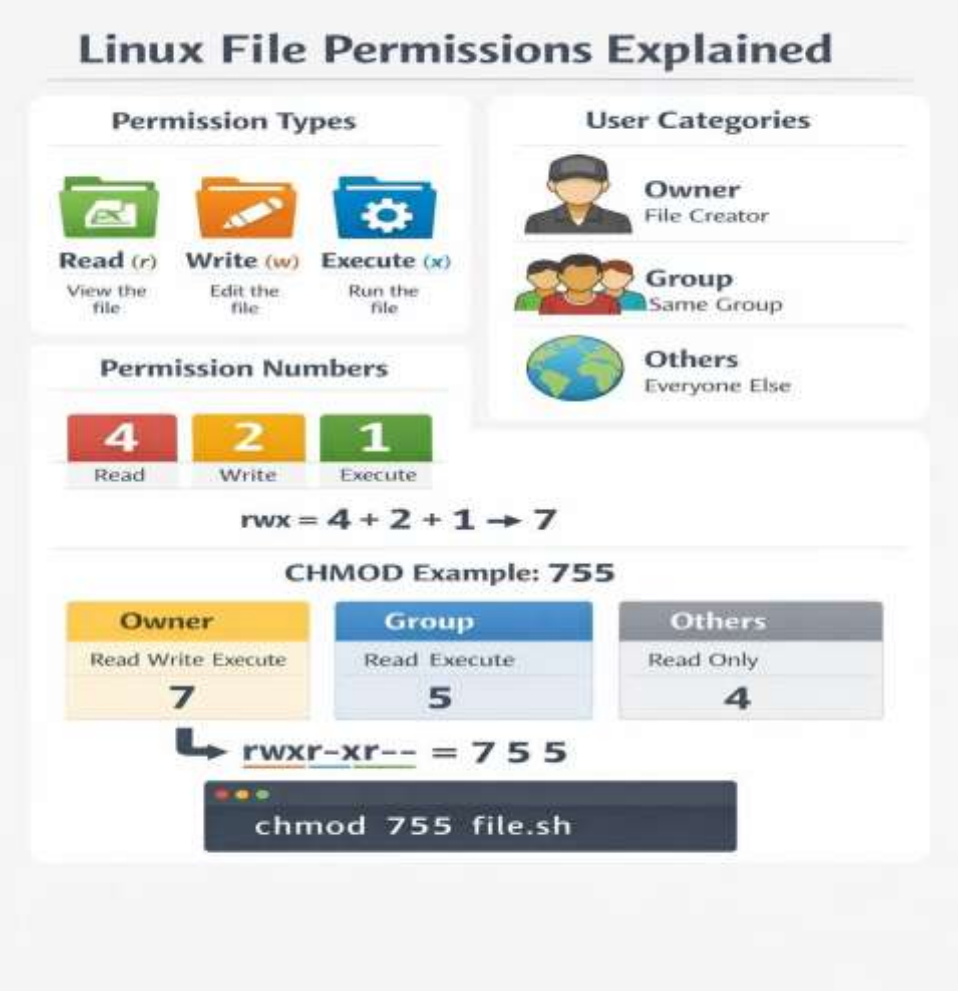


chmod u+x file1
Only **Akshansh** is affected.

chmod g+x file1
Only the **Linux Students** group is affected.

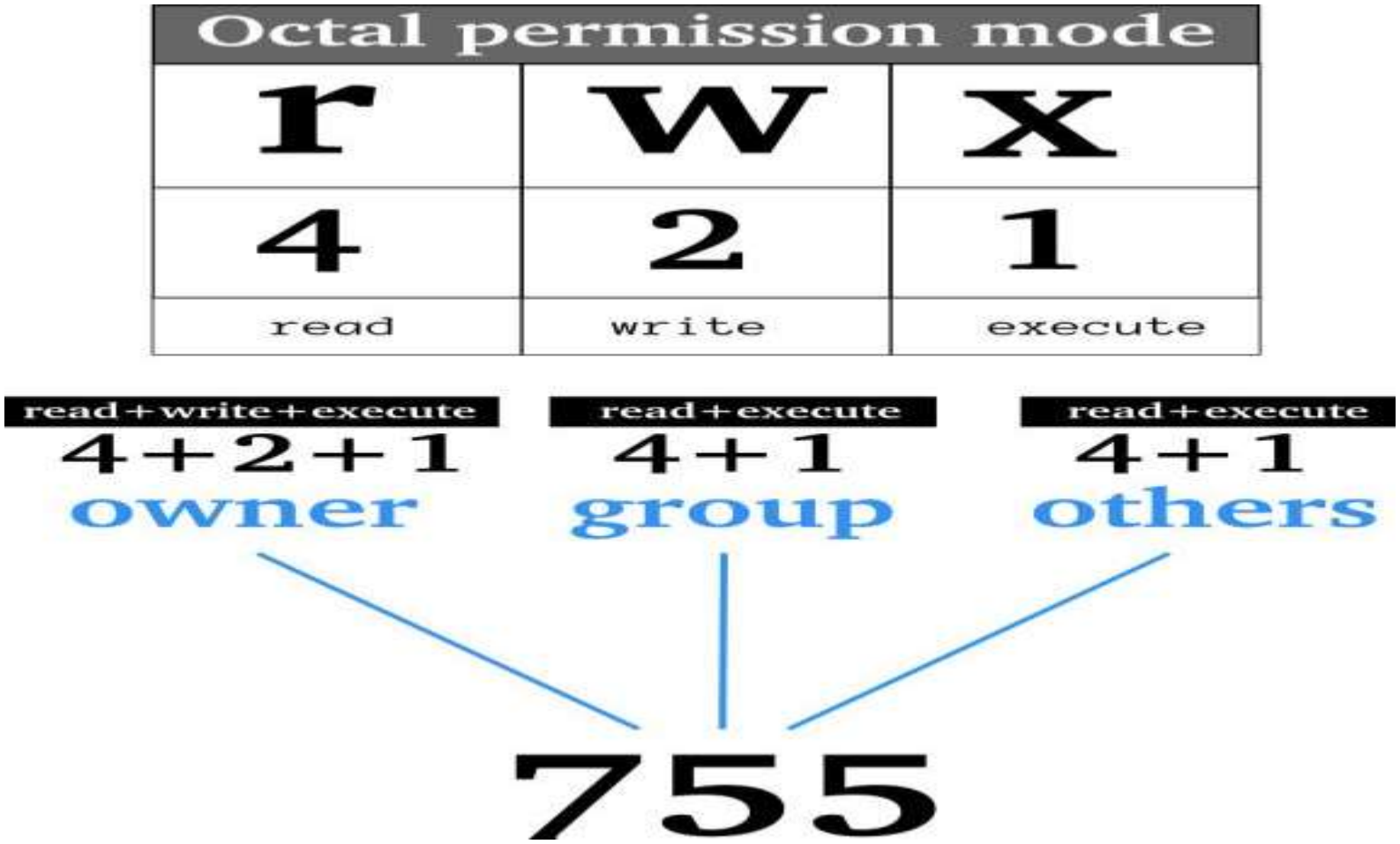
chmod o+x file1
Only everyone else is affected

Managing Files permission Numerically

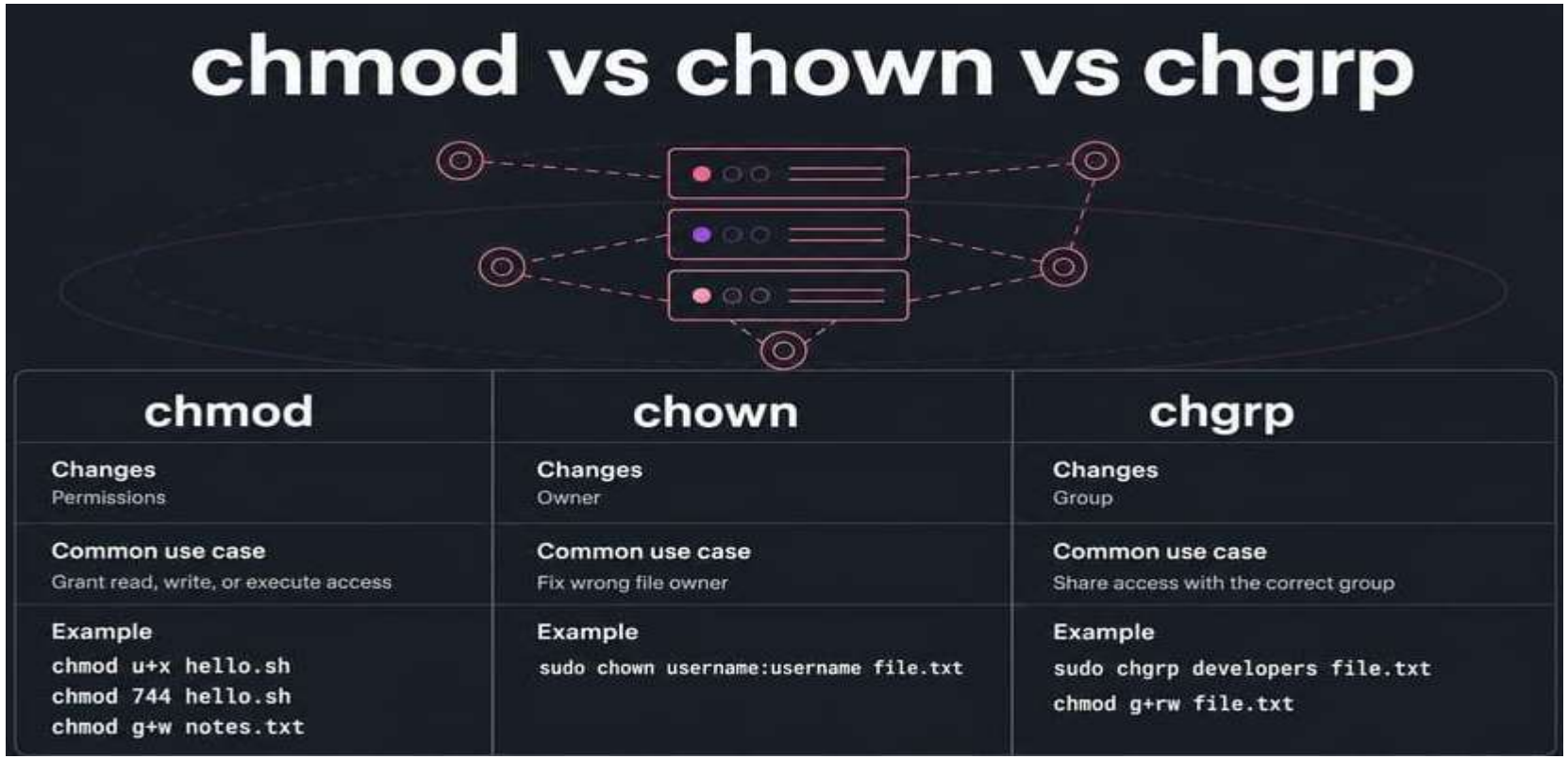


Permission	Calculation	Number
---	0	**0**
--x	1	**1**
-w-	2	**2**
-wx	2 + 1	**3**
r--	4	**4**
r-x	4 + 1	**5**
rw-	4 + 2	**6**
rwx	4 + 2 + 1	**7**

CONTROLLING ACCESS TO FILES

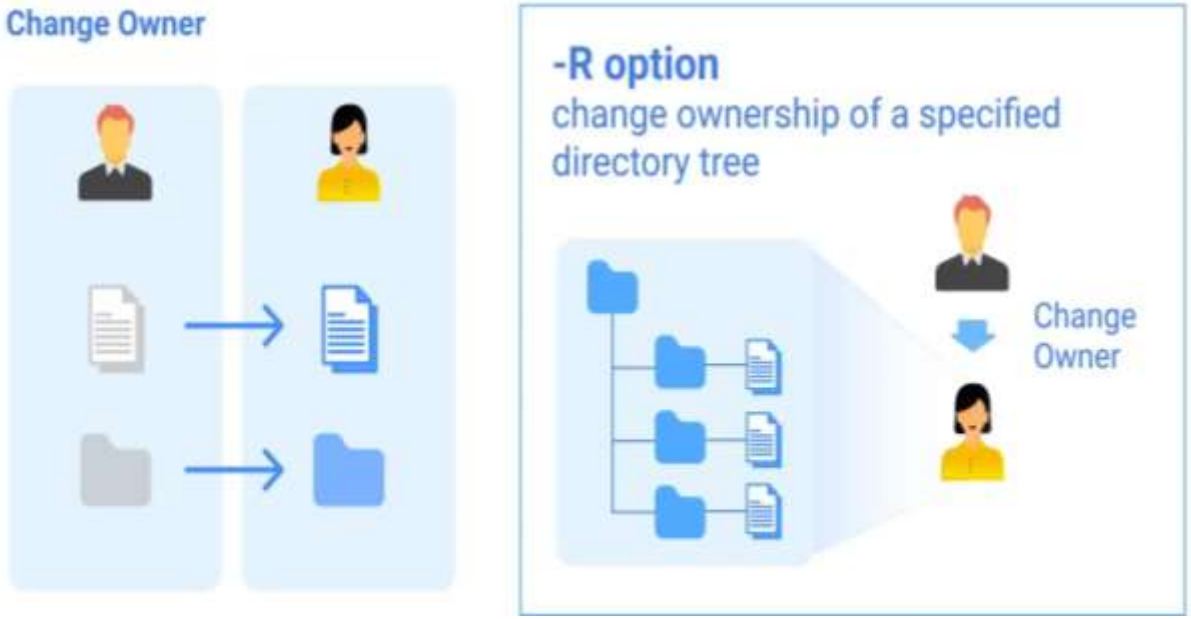


CONTROLLING ACCESS TO FILES



CONTROLLING ACCESS TO FILES

chown [*user name*] [*file or directory path*]



Recursive — (-R)
change ownership of the directory AND everything inside it.

CONTROLLING ACCESS TO FILES

Cross-Group File Access

There are two teams on the Ubuntu server: A,B

Task 1 — Create Users
Task 2 — Create Groups
Task 3 — Assign Users
Task 4 — Create Project File
Task 5 — Configure Ownership
Task 6 — Restrict the File
Task 7 — Test A ‘s Access
Task 8 — Give A’s Access Through Group Membership

CONTROLLING ACCESS TO FILES

MANAGING DEFAULT PERMISSIONS AND FILE ACCESS

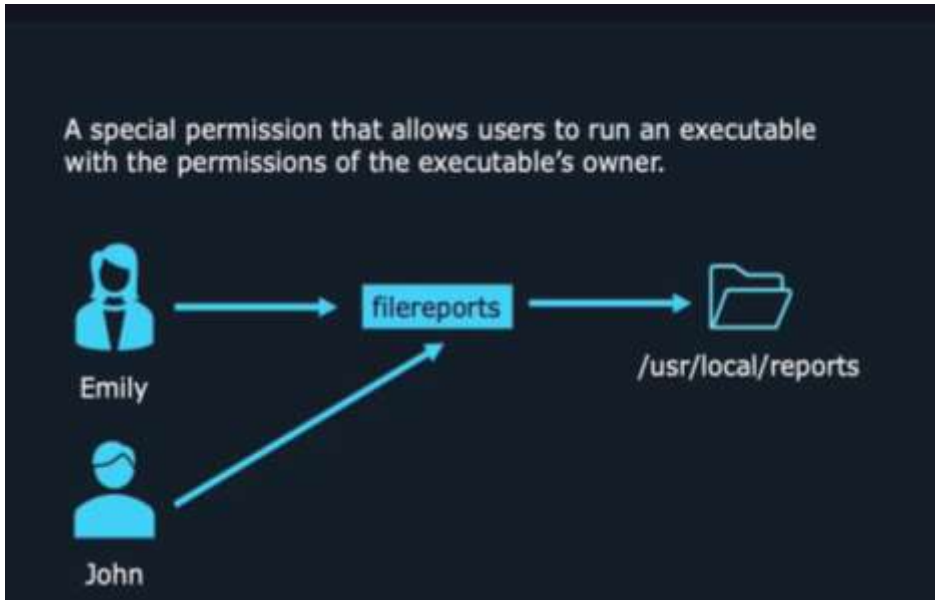
Special permissions constitute a fourth permission type in addition to the basic user, group, and other types. As the name implies, these permissions provide additional access-related features over and above what the basic permission types allow

SPECIAL PERMISSION	EFFECT ON FILES	EFFECT ON DIRECTORIES
u+s (suid)	File executes as the user that owns the file, not the user that ran the file.	No effect.
g+s (sgid)	File executes as the group that owns the file.	Files newly created in the directory have their group owner set to match the group owner of the directory.
o+t (sticky)	No effect.	Users with write access to the directory can only remove files that they own; they cannot remove or force saves to files owned by other users.

CONTROLLING ACCESS TO FILES

MANAGING DEFAULT PERMISSIONS AND FILE ACCESS

SETUID = Run an executable with the permissions of its owner



WITHOUT SETUID

Admin owns the program

change-password

Owner = Admin

Now **User** wants to use it:

User

runs

change-password

Normally, the program runs using **User permissions**.

The problem

User may not have permission to change certain protected information.

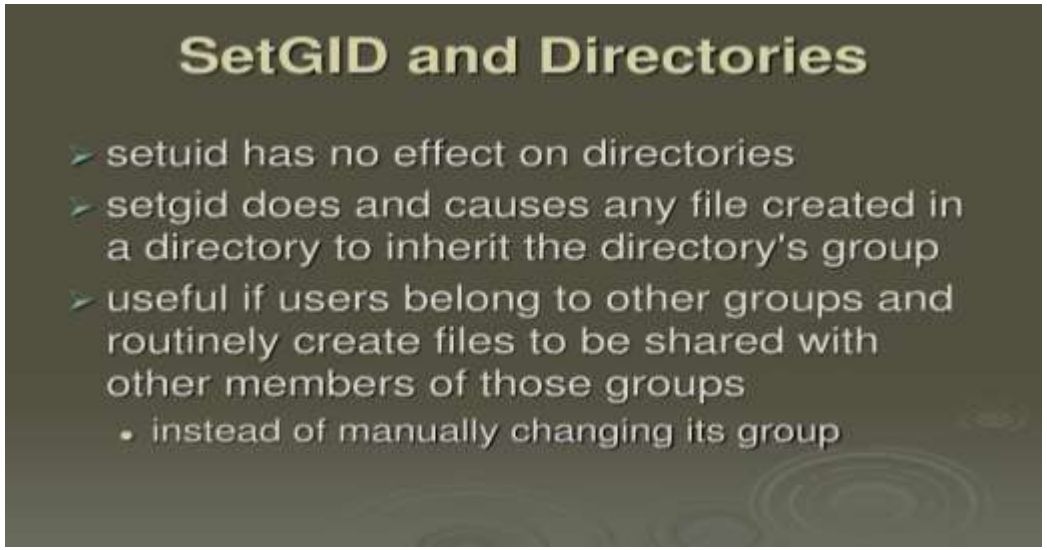
So how can an ordinary user safely use a program that needs special privileges?

SETUID solves this problem.

CONTROLLING ACCESS TO FILES

MANAGING DEFAULT PERMISSIONS AND FILE ACCESS

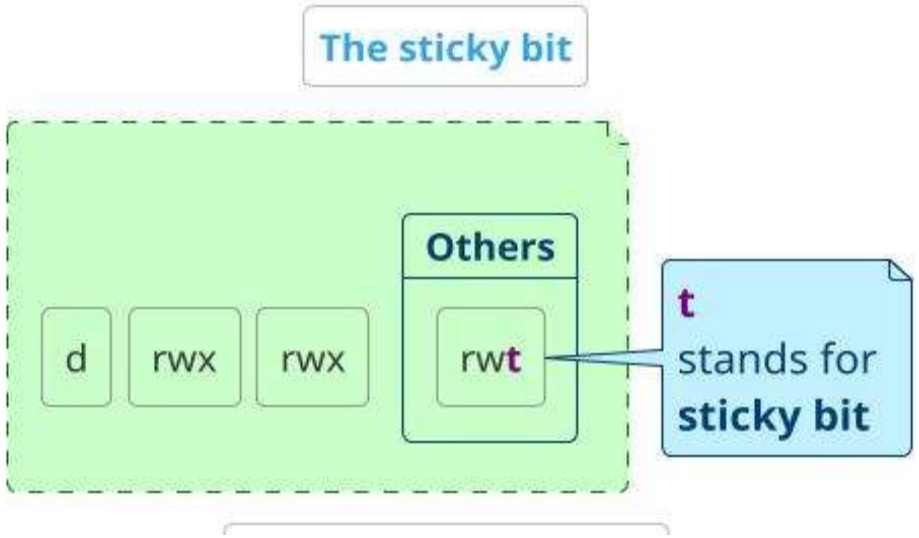
SETGID : "STAY WITH THE GROUP"



Imagine a college project team.**U1,U2,U3**
They all belong to the same team:**AI-Team**
They have a shared folder:
PROJECT/
The folder belongs to:
Group = AI-Team
The problem
U1 creates:
report.txt
Who should the file belong to?
We want:
Owner = U1
Group = AI-Team
Why?Because U2 and U2 are also members of **AI-Team**. **SETGID helps us do this automatically.**

CONTROLLING ACCESS TO FILES

MANAGING DEFAULT PERMISSIONS AND FILE ACCESS



WITHOUT STICKY BIT

Everyone has permission to use the shared folder.ie u1,u2,u3

SHARED/

- u1.txt
- u2.txt
- u3.txt

u3 can access the shared directory.

If the directory permissions allow users to delete entries, u3 may be able to remove:

u1.txt

even though:

Owner = u1

Command : chmod o+t SHARED

MONITORING AND MANAGING LINUX PROCESSES

DEFINITION OF A PROCESS

A process is a running instance of a launched, executable program.

A process consists of:

- An address space of allocated memory
- Security properties including ownership credentials and privileges
- One or more execution threads of program code
- Process state

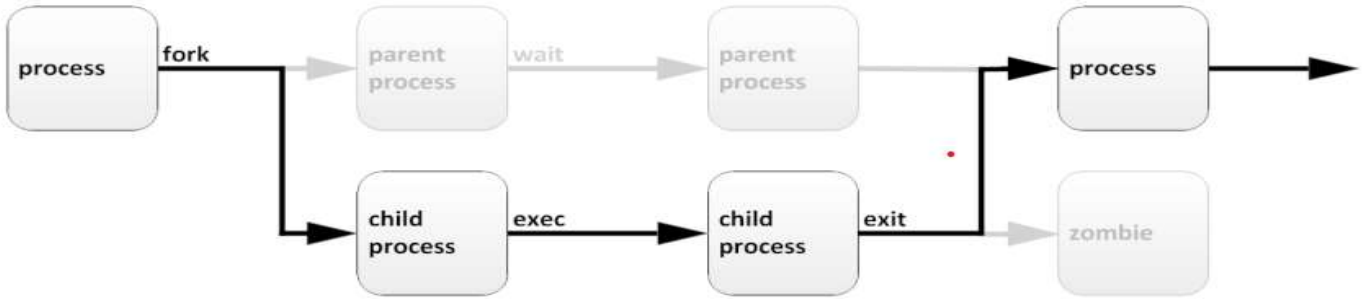
The environment of a process includes:

- Local and global variables
- A current scheduling context
- Allocated system resources, such as file descriptors and network ports

MONITORING AND MANAGING LINUX PROCESSES

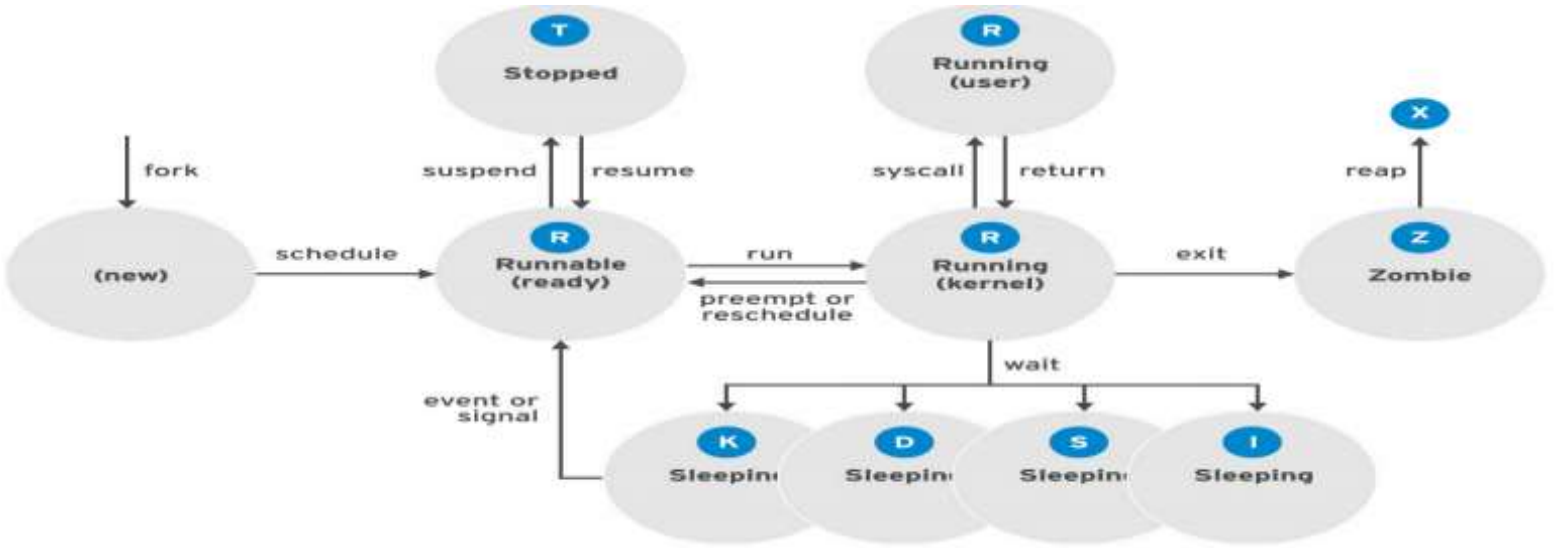
An existing (parent) process duplicates its own address space (**fork**) to create a new (child) process structure. Every new process is assigned a unique process ID (PID) for tracking and security. The PID and the parent's process ID (PPID) are elements of the new process environment. Any process may create a child process.

Through the fork routine, a child process inherits security identities, previous and current file descriptors, port and resource privileges, environment variables, and program code. A child process may then exec its own program code. Normally, a parent process sleeps while the child process runs, setting a request (wait) to be signalled when the child completes. Upon exit, the child process has already closed or discarded its resources and environment. The only remaining resource, called a zombie, is an entry in the process table



MONITORING AND MANAGING LINUX PROCESSES

In a multitasking operating system, each CPU (or CPU core) can be working on one process at a single point in time. As a process runs, its immediate requirements for CPU time and resource allocation change. Processes are assigned a state, which changes as circumstances dictate.



A **zombie process** is a terminated child process that remains in the system's process table because its parent process has not yet read its exit status. It uses no CPU or memory, but holds a Process ID (PID) slot

NAME	FLAG	KERNEL-DEFINED STATE NAME AND DESCRIPTION
Running	**R**	TASK_RUNNING: The process is either executing on a CPU or waiting to run. Process can be executing user routines or kernel routines (system calls), or be queued and ready when in the *Running* (or *Runnable*) state.
Sleeping	**S**	TASK_INTERRUPTIBLE: The process is waiting for some condition: a hardware request, system resource access, or signal. When an event or signal satisfies the condition, the process returns to *Running*.
	D	TASK_UNINTERRUPTIBLE: This process is also *Sleeping*, but unlike **S** state, does not respond to signals. Used only when process interruption may cause an unpredictable device state.
	K	TASK_KILLABLE: Identical to the uninterruptible **D** state, but modified to allow a waiting task to respond to the signal that it should be killed (exit completely). Utilities frequently display *Killable* processes as **D** state.

NAME	FLAG	KERNEL-DEFINED STATE NAME AND DESCRIPTION
	I	TASK_REPORT_IDLE: A subset of state **D**. The kernel does not count these processes when calculating load average. Used for kernel threads. Flags TASK_UNINTERRUPTABLE and TASK_NOLOAD are set. Similar to TASK_KILLABLE, also a subset of state **D**. It accepts fatal signals.
Stopped	**T**	TASK_STOPPED: The process has been *Stopped* (suspended), usually by being signaled by a user or another process. The process can be continued (resumed) by another signal to return to *Running*.
	T	TASK_TRACED: A process that is being debugged is also temporarily *Stopped* and shares the same **T** state flag.
Zombie	**Z**	EXIT_ZOMBIE: A child process signals its parent as it exits. All resources except for the process identity (PID) are released.
	X	EXIT_DEAD: When the parent cleans up (*reaps*) the remaining child process structure, the process is now released completely. This state will never be observed in process-listing utilities.

Why Process States are Important

When troubleshooting a system, it is important to understand how the kernel communicates with processes and how processes communicate with each other. At process creation, the system assigns the process a state. The **S** column of the **top** command or the **STAT** column of the **ps** show the state of each process. On a single CPU system, only one process can run at a time. It is possible to see several processes with a state of **R**. However, not all of them will be running consecutively, some of them will be in status waiting.

```
[user@host ~]$ top
PID USER   PR   NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  1 root   20    0  244344  13684   9024 S   0.0   0.7   0:02.46 systemd
  2 root   20    0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
...output omitted...
```

The top command is a real-time system monitoring utility available on Linux systems. It presents a continuously updated view of running processes along with overall system resource usage such as CPU load, memory consumption, and system uptime

LISTING PROCESSES

The **ps** command is used for listing current processes. It can provide detailed process information, including:

- User identification (UID), which determines process privileges
- Unique process identification (PID)
- CPU and real time already expended
- How much memory the process has allocated in various locations
- The location of process **stdout**, known as the controlling terminal
- The current process state.

The ps aux command in Linux is used to display a **comprehensive, static snapshot of all currently running processes** on the system. It is one of the most widely used tools by system administrators for monitoring resource usage, troubleshooting system health, and identifying process

```
[user@host ~]$ ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.1  0.1  51648  7504 ?        Ss   17:45   0:03 /usr/lib/systemd/
syst
root         2  0.0  0.0      0     0 ?        S    17:45   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    17:45   0:00 [ksoftirqd/0]
root         5  0.0  0.0      0     0 ?        S<   17:45   0:00 [kworker/0:0H]
root         7  0.0  0.0      0     0 ?        S    17:45   0:00 [migration/0]
```

Controlling Jobs

DESCRIBING JOBS AND SESSIONS

Job control is a feature of the shell which allows a single shell instance to run and manage multiple commands.

Types of Jobs :
A **foreground job** is a job that is **currently running in the terminal and has control of the keyboard**
Foreground = The job currently using the terminal.

A **background job** is a job that **runs without controlling the keyboard**, allowing you to continue using the terminal for other commands

Examples
Foreground Job Example: Running sleep 100 or sudo apt update. The terminal stays busy, shows the progress or waits, and does not let you type a new command until it is done or stopped
Background Job Example: Running sleep 500 &. Adding the & symbol tells the system to run the sleep timer in the background, immediately returning your command prompt so you can do other work.

Controlling Jobs

Cron Job: Cron is a time-based job scheduling utility available in Linux and Unix-like operating systems. It runs as a background daemon process and automatically executes scheduled tasks at predefined times without user intervention

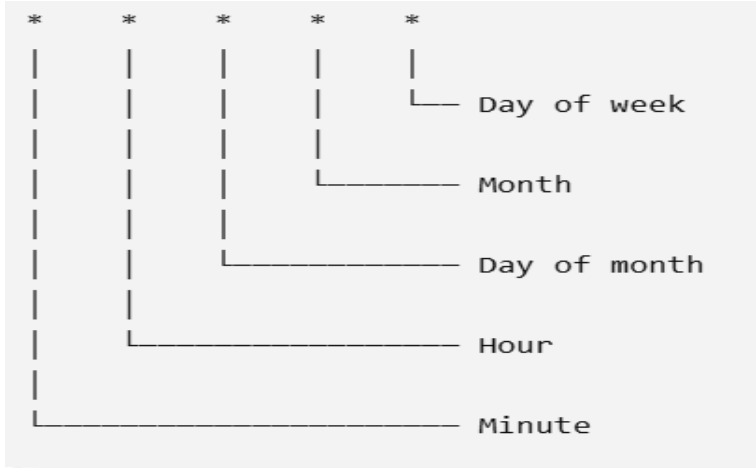
Working of Cron :

Cron operates as a background service that continuously monitors scheduled jobs and executes them when their defined time matches the system time

	Command	Purpose	Example
1	crontab -e	**Create or edit** your cron jobs	crontab -e
2	crontab -l	**List/view** your existing cron jobs	crontab -l
3	crontab -r	**Remove all** cron jobs for the current user	crontab -r
4	crontab -i	**Remove all** cron jobs with confirmation	crontab -i
5	crontab -u username -e	**Edit another user's** crontab (requires appropriate privileges)	sudo crontab -username -e

Creating a Cron Job demonstration :

Step 1 — Create a new crontab
crontab –e
Your editor will open.At the bottom, add **only this line**:
* * * * * echo "Cron is working" >> /tmp/cron_test.txt



Step 2 - Verify the job exists
crontab –l

Step 3 - Wait 1–2 minutes
Then run : cat /tmp/cron_test.txt
cron's basic job is simply:At a specified time, automatically execute this command.

Step 4 crontab –r
To remove job

KILLING PROCESSES

PROCESS CONTROL USING SIGNALS

A signal is a software interrupt delivered to a process. Each signal has a default action, usually one of the following:

- Term - Cause a program to terminate (exit) at once.
- Core - Cause a program to save a memory image (core dump), then terminate.
- Stop - Cause a program to stop executing (suspend) and wait to continue (resume).

SIGNAL NUMBER	SHORT NAME	DEFINITION	PURPOSE
1	HUP	Hangup	Used to report termination of the controlling process of a terminal. Also used to request process reinitialization (configuration reload) without termination.
2	INT	Keyboard interrupt	Causes program termination. Can be blocked or handled. Sent by pressing INTR key combination (**Ctrl+C**).
3	QUIT	Keyboard quit	Similar to SIGINT, but also produces a process dump at termination. Sent by pressing QUIT key combination (**Ctrl+**).
9	KILL	Kill, unblockable	Causes abrupt program termination. Cannot be blocked, ignored, or handled; always fatal.
15 *default*	TERM	Terminate	Causes program termination. Unlike SIGKILL, can be blocked, ignored, or handled. The "polite" way to ask a program to terminate; allows self-cleanup.
18	CONT	Continue	Sent to a process to resume, if stopped. Cannot be blocked. Even if handled, always resumes the process.

KILLING PROCESSES

The **kill** command sends a signal to a process by PID number. Despite its name, the kill command can be used for sending any signal, not just those for terminating programs. You can use the **kill -l** command to list the names and numbers of all available signals

```
[user@host ~]$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
...output omitted...
[user@host ~]$ ps aux | grep job
5194  0.0  0.1 222448  2980 pts/1    S    16:39   0:00 /bin/bash /home/user/bin/
control job1
5199  0.0  0.1 222448  3132 pts/1    S    16:39   0:00 /bin/bash /home/user/bin/
control job2
5205  0.0  0.1 222448  3124 pts/1    S    16:39   0:00 /bin/bash /home/user/bin/
control job3
5430  0.0  0.0 221860  1096 pts/1    S+   16:41   0:00 grep --color=auto job
```

KILLING PROCESSES

The **kill** command sends a signal to a process by PID number. Despite its name, the kill command can be used for sending any signal, not just those for terminating programs. You can use the **kill -l** command to list the names and numbers of all available signals

```
[user@host ~]$ kill 5194
[user@host ~]$ ps aux | grep job
user    5199  0.0  0.1 222448  3132 pts/1    S    16:39   0:00 /bin/bash /home/
user/bin/control job2
user    5205  0.0  0.1 222448  3124 pts/1    S    16:39   0:00 /bin/bash /home/
user/bin/control job3
user    5783  0.0  0.0 221860   964 pts/1    S+   16:43   0:00 grep --color=auto
 job
[1]   Terminated              control job1
[user@host ~]$ kill -9 5199
[user@host ~]$ ps aux | grep job
user    5205  0.0  0.1 222448  3124 pts/1    S    16:39   0:00 /bin/bash /home/
user/bin/control job3
user    5930  0.0  0.0 221860  1048 pts/1    S+   16:44   0:00 grep --color=auto
 job
[2]-  Killed                  control job2
[user@host ~]$ kill -SIGTERM 5205
user    5986  0.0  0.0 221860  1048 pts/1    S+   16:45   0:00 grep --color=auto
 job
[3]+  Terminated              control job3
```